

THE NEW SCHOOL • EUGENE LANG COLLEGE OF LIBERAL ARTS

Data and Design with Python

LMTH 2080

Fall 2026 • Mondays, 9:00–11:40 AM

Eugene Lang College of Liberal Arts

Instructor Jacob Koehler
Office Hours By appointment

Email koehlerj@newschool.edu
Course Meeting Mondays, 9:00–11:40 AM

Course Description

Data is everywhere, but having data is not the same as having useful data. This course introduces Python through the problems that data can help us investigate, explain, and address. We will begin by asking what questions matter to us, what evidence might help answer them, where that evidence comes from, and what may be missing from the data we find.

Python will be our primary computational medium. Students will learn to organize computational projects, write and debug programs, work with tabular data, acquire data from the web, create visualizations, and build interactive applications. Later in the semester, we will study basic ideas from machine learning and neural networks before working with pretrained models from Hugging Face.

The course culminates in the design and deployment of an interactive Streamlit application built around a pretrained model. Throughout the semester, our emphasis will be not only on whether code works, but whether the data, model, representation, and interface are useful for the problem and people for whom they are intended.

No prior experience with Python is required. Students are expected to have access to a laptop computer and bring it to every class.

Key Idea

The central question of the course is not simply “What can we do with Python?” but *What are we trying to understand or change, and what data and computational tools would actually help?*

Learning Outcomes

By the successful completion of this course, students will be able to:

1. formulate questions that can be investigated with data and evaluate whether a dataset is useful for those questions;
2. navigate a filesystem from the command line and use Git to record and manage changes to a computational project;
3. write, read, test, and debug Python programs using variables, data types, collections, conditionals, loops, and functions;
4. use pandas to load, inspect, clean, transform, summarize, and analyze tabular data;

5. create and critique data visualizations with attention to visual encoding, audience, uncertainty, and purpose;
6. acquire and structure data from web APIs and other online sources;
7. build interactive data applications using Streamlit;
8. explain the basic logic of supervised machine learning, including features, targets, training and test data, prediction, evaluation, and overfitting;
9. distinguish model training from inference and explain at a conceptual level how pretrained models are used;
10. evaluate a pretrained model using its documentation, intended uses, inputs and outputs, limitations, and observed failure cases; and
11. design, build, document, and present an interactive application that makes thoughtful use of data and a pretrained model.

Course Questions

We will return to a small set of questions throughout the semester:

Data Question

What problem or question are you interested in? What would you need to know to investigate it? What data could provide that evidence? Who collected the data, for what purpose, and what does it fail to capture?

Design Question

Who is this for? What should they be able to see, understand, decide, or do? How do the choices we make in code, representation, and interface shape that experience?

Projects and Assignments

The semester is organized around three increasingly public forms of computational work:

Project I: Data Investigation

Students will identify or work with a dataset, clean and transform it, ask questions of it, and communicate findings through analysis and visualization. The primary artifact will be a well-organized computational notebook.

Project II: Designed Data Application

Students will build a small Streamlit application around a dataset or data source. The emphasis is on moving from analysis performed for oneself to an interactive object designed for another person.

Final Project: Pretrained Model Application

Students will select a pretrained model from the Hugging Face Model Hub and design an interactive Streamlit application around it. Students will be expected to understand the model well enough to describe its purpose, inputs, outputs, documentation, known limitations, and observed failure cases. The final project will be presented in class.

Notebook → Data Application → Model Application

Shorter assignments, readings, coding exercises, and in-class studio work will support these projects.

Evaluation

Component	Weight
Active Participation / Attendance	30%
Homework and Short Assignments	40%
Projects	30%
Total	100%

Projects will be evaluated not only on technical correctness. Depending on the assignment, evaluation may also consider the quality of the question, choice and understanding of data, clarity of analysis, design decisions, documentation, testing, communication, and reflection.

Working With Generative AI

Generative AI systems can produce code, explanations, and interfaces, but using generated code effectively requires being able to read it, reason about it, test it, and recognize when it is wrong.

Early in the semester, some assignments may restrict the use of generative AI so that everyone develops a working foundation in Python and debugging. As the course progresses—and particularly as we study machine learning, transformers, and pretrained models—we will examine these tools more directly and use them where appropriate.

When AI-assisted tools are permitted, students remain responsible for every part of the work they submit. You should be able to explain your code, modify it, test it, identify its assumptions, and describe how AI tools contributed to the work. Assignment-specific instructions take precedence over this general policy.

Materials and Computing

Students should bring a laptop to every class. We will use a Unix-like command line, Git and GitHub, Python, Jupyter, pandas, visualization libraries, Streamlit, scikit-learn, and Hugging Face tools.

Software used in the course is freely available. Installation instructions and course-specific setup guidance will be provided. Students using Windows may need to install additional tools to obtain the command-line environment used in class.

Readings will include official documentation and short selections on data, visualization, design, computation, machine learning, and model documentation. Additional readings will be distributed through the course site or

Canvas.

Course Calendar

The calendar describes the planned arc of the semester and may change in response to the pace and interests of the class. Changes will be announced in class and through the course site or Canvas.

Date	Topic	Questions and Activities
Aug. 31	Useful Data	What problems are interesting to us? What data could help? Data hunt; datasets as evidence; what is missing? Introduction to files, the Unix shell, Git, and GitHub.
Sep. 7	No Class	Labor Day.
Sep. 14	Python as a Tool	Jupyter; expressions, variables, strings, numbers, Boolean values, and computational questions about data.
Sep. 21	Programming with Data	Collections, conditionals, iteration, functions, decomposition, and debugging.
Sep. 28	Data as Material	pandas; Series and DataFrames; reading CSV files; selecting, filtering, and inspecting data. Project I introduced.
Oct. 5	Transforming Data	Missing values, types, strings, dates, grouping, aggregation, and the choices involved in cleaning data.
Oct. 12	Seeing Data	Visualization; visual encoding; exploratory and explanatory graphics; asking what a visualization makes visible.
Oct. 19	Getting Data	HTTP, APIs, JSON, requests, public data sources, and light web data acquisition. Project I due.
Oct. 26	Designing with Data	Critique, annotation, narrative, uncertainty, audience, and visual argument. Project II introduced.
Nov. 2	Interactive Data	Streamlit fundamentals; widgets, layout, interaction, and designing an interface around data.
Nov. 9	Models and Prediction I	Features and targets; training and test data; regression, classification, and scikit-learn.
Nov. 16	Models and Prediction II	Evaluation, prediction, overfitting, failure, and model limitations. Project II due.
Nov. 23	Neural Networks and Pretrained Models	Training versus inference; neural networks, embeddings, and transformers at a conceptual level. Final project introduced.
Nov. 30	Hugging Face	The Model Hub, model cards, pipelines, selecting models, testing inputs and outputs, and examining limitations.
Dec. 7	Designing AI Applications	Hugging Face + Streamlit; prototype critique, interface design, failure modes, testing, and final-project studio.
Dec. 14	Final Presentations	Presentation and critique of pretrained-model Streamlit applications.

Community Agreements

Community agreements are classroom expectations for discussion and behavior that we will create collaboratively. They provide a foundation for an inclusive learning environment through shared expectations, transparency,

and responsibility. We will establish our agreements early in the semester and revisit them as needed.

Course Policies

Responsibility

Students are responsible for assignments and course material even when absent. If you miss class, consult the course site and communicate with the instructor or classmates about what you missed. Because much of our work is cumulative, falling behind on technical material can quickly make later work more difficult.

Participation

This is a studio-like computational course. Participation includes coming to class prepared and on time, contributing to discussions and critiques, working productively with classmates, attempting in-class problems, asking questions, helping create a productive classroom environment, and engaging with the process of writing and debugging code.

Canvas and Course Materials

Canvas and the course website will be used for announcements, assignments, readings, links, and course materials. Students should check for updates before each class meeting.

Attendance

At Lang College, regular attendance and participation are important because learning is both individual and collective. For a course meeting once per week, the Lang attendance policy calls for a conversation among the student, instructor, and Student Success Advisor after two absences; more than three absences normally requires withdrawal or a failing grade, subject to an agreed attendance plan and applicable university policies.

Absences are counted from the first day a student is enrolled. Students should communicate with the instructor when circumstances affect their ability to attend or participate.

Academic Integrity

Students are responsible for understanding The New School's policies on academic honesty and integrity and for appropriately distinguishing their own work from the work of others. This applies to code and computational work as well as writing, research, presentations, images, and other submitted materials.

Academic integrity in a programming course includes accurately representing the sources of code and assistance, following assignment-specific rules about collaboration and AI-assisted tools, and being able to explain work submitted as your own.

See The New School University Policies and the University Learning Center for the current university policy and guidance.

University Resources

The New School provides resources including University Libraries, the University Learning Center, Student Disability Services, Archives and Special Collections, food assistance, health and wellness services, the Student Ombuds Office, and Financial Aid.

The University Learning Center provides support in areas including software, computer programming, writing, mathematics, oral presentations, and time management. Students who need disability-related accommodations should work with Student Disability Services through the university's established process.

Current links, contact information, and university policy language are available through The New School website and Canvas.

A Note on the Calendar

The topics, project dates, and readings in this syllabus represent our plan for the semester. Data and programming rarely proceed exactly according to plan, and neither does a good course. We may adjust the schedule to spend more time on ideas that require it, respond to student interests, or incorporate useful datasets and tools that emerge during the semester. Any substantive changes will be communicated clearly.